```
begin


using DifferentialEquations
using ForwardDiff
using WGLMakie
using LaTeXStrings
using PlutoUI

end
```

constants are defined here

```
md"""
### constants are defined here
"""
```

```
(I_ext = 10.0, g_Na = 120.0, g_K = 36.0, g_L = 0.3, E_Na = 50.0, E_K = -77.0, E_L = -54.4, C =
```

```
begin
    # Rates for Sodium Activation (m)
    # Fix: If V == -40, the value is 1.0 (the limit)
    α_m(V) = abs(V + 40.0) < 1e-7 ? 1.0 : (0.1 * (V + 40.0)) / (1.0 - exp(-(V + 40.0) / 10.0))
    β_m(V) = 4.0 * exp(-(V + 65.0) / 18.0)

    # Rates for Sodium Inactivation (h)
    α_h(V) = 0.07 * exp(-(V + 65.0) / 20.0)
    β_h(V) = 1.0 / (1.0 + exp(-(V + 35.0) / 10.0))

    # Rates for Potassium Activation (n)
    # Fix: If V == -55, the value is 0.1 (the limit)
    α_n(V) = abs(V + 55.0) < 1e-7 ? 0.1 : (0.01 * (V + 55.0)) / (1.0 - exp(-(V + 55.0) / 10.0))
    β_n(V) = 0.125 * exp(-(V + 65.0) / 80.0)

    # Parameters
p = (
    I_ext = 10.0,  # External current (μA/cm²) - Try changing this to 2.0 or 20.0!
    g_Na  = 120.0, # Max Sodium conductance (mS/cm²)
    g_K   = 36.0,  # Max Potassium conductance (mS/cm²)
    g_L   = 0.3,   # Leak conductance (mS/cm²)
    E_Na  = 50.0,  # Sodium reversal potential (mV)
    E_K   = -77.0, # Potassium reversal potential (mV)
    E_L   = -54.4, # Leak reversal potential (mV)
    C     = 1.0    # Membrane capacitance (μF/cm²)
)

end
```

```
Hodgkin_Huxley! (generic function with 1 method)
```

```julia
function Hodgkin_Huxley!(du, u , p , t)
    V,m,h,n = u
    I_ext, g_Na, g_K, g_L, E_Na, E_K, E_L, C = p

    # calculating ionic currents
    I_Na = g_Na * (m^3) * h * ( V - E_Na)
    I_K = g_K * (n^4) * ( V - E_Na)
    I_L = g_L * ( V - E_L)


    # MEMBRANE POTENTIAL DIFFERRENTIAL EQUATIONS
    du[1] = (I_ext - I_Na - I_K - I_L) / C

    # GATING VARIABLE DIFFERENTIAL EQUATIONS (dx/dt = α(1-x) - βx)

    du[2] = α_m(V) * (1 - m) - β_m(V) * m
    du[3] = α_h(V) * (1 - h) - β_h(V) * h
    du[4] = α_n(V) * (1 - n) - β_n(V) * n
end

```

Error message

MethodError: no method matching promote_tspan(::@NamedTuple{I_ext::Float64, g_Na::Float64, g_K::Float64, g_L::Float64, E_Na::Float64, E_K::Float64, E_L::Float64, C::Float64})
The function `promote_tspan` exists, but no method is defined for this combination of argument types.

Closest candidates are:
  promote_tspan(::Any, ::Any, ::Any, ::Any, ::Any)
   @ DiffEqBase C:\Users\nirbh\.julia\packages\DiffEqBase\bcYrc\src\solve.jl:747
  promote_tspan(::Nothing)
   @ SciMLBase C:\Users\nirbh\.julia\packages\SciMLBase\MzDs3\src\problems\problem_utils.jl:9
  promote_tspan(::AbstractArray{<:ForwardDiff.Dual}, ::Any, ::Tuple{ForwardDiff.Dual, ForwardDiff.Dual}, ::Any, ::Any)
   @ DiffEqBaseForwardDiffExt C:\Users\nirbh\.julia\packages\DiffEqBase\bcYrc\ext\DiffEqBaseForwardDiffExt.jl:162
  ...

Show stack trace...

```
begin
    # Intial conditions : [V,m.h,n]
    u0 = [-65.0,0.05, 0.6, 0.32]
    tspan = (0.0,50.0)

    # set the problem then solving it
    prob = ODEProblem(Hodgkin_Huxley!,u0,p,tspan)
    sol = solve(prob, Tsit5(),reltol = 1e-6, abstol = 1e-6)
end
```